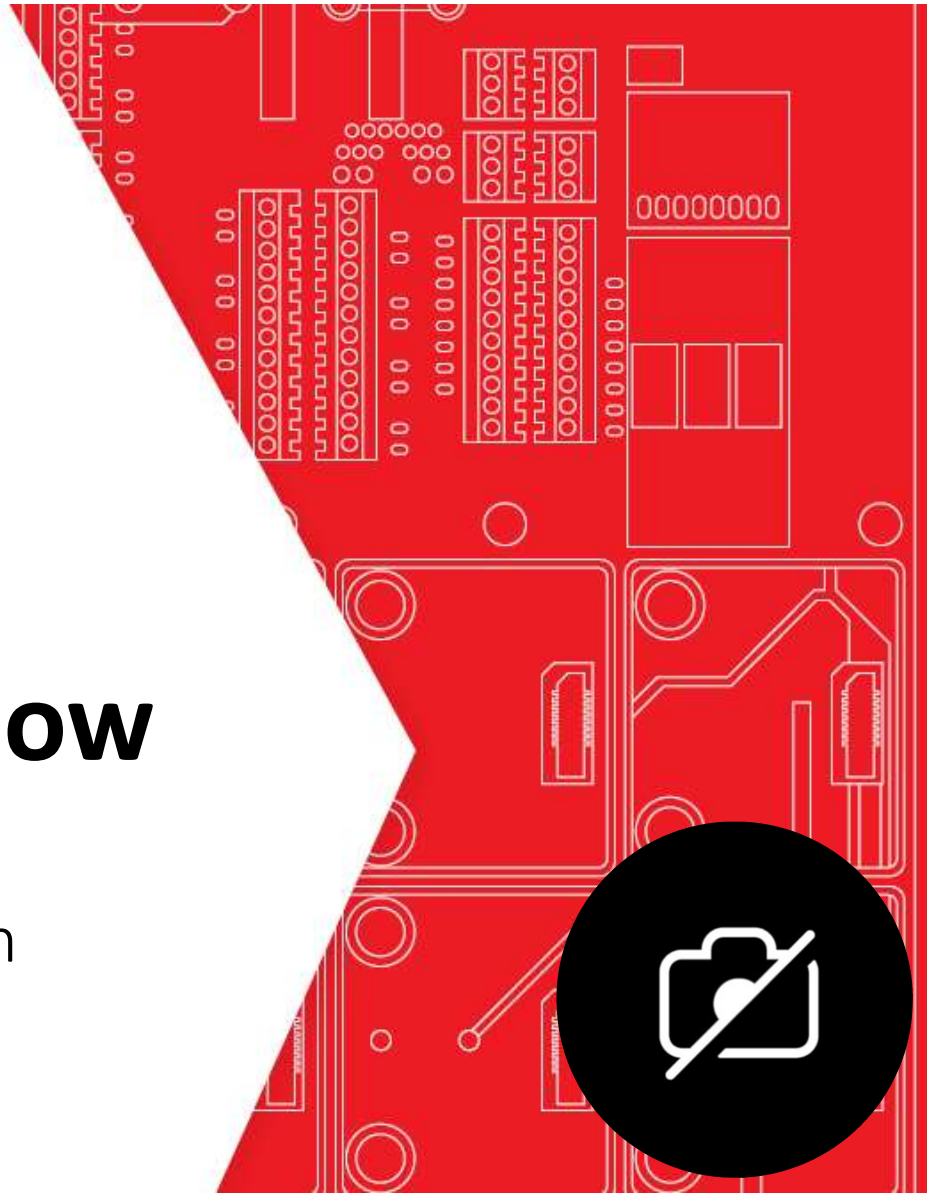
Adding AI to your Hardware System Development Workflow

Zach Olson, LabVIEW Champion

Cyth Systems

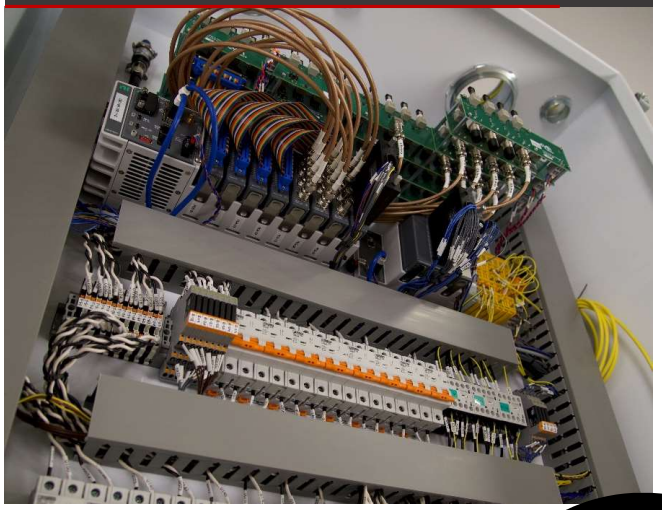


Evolving system design and validation

Use more AI tools

- Speed of code generation
- Leveraged industry knowledge
- Do more with less

Reliable precision embedded systems

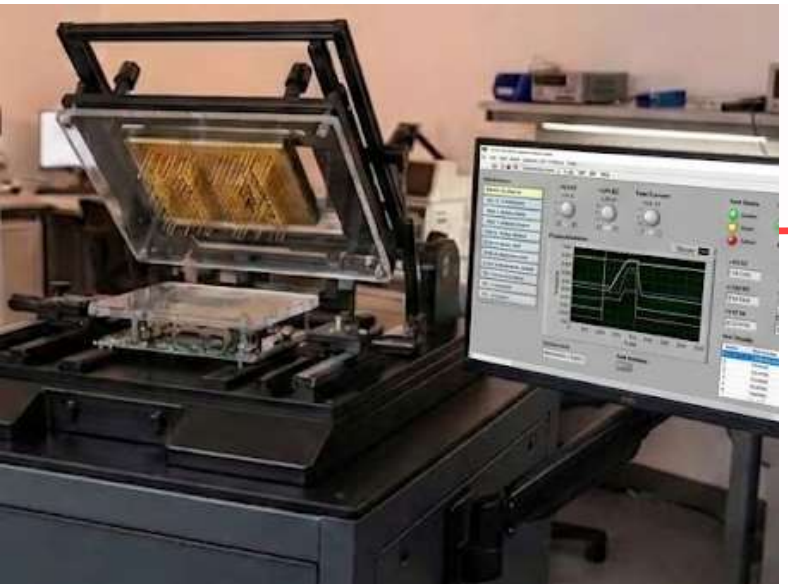


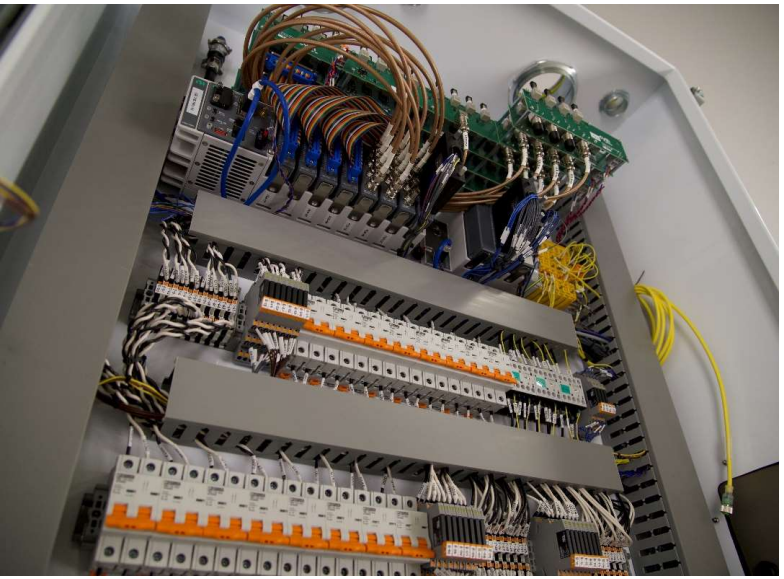
- Determinism and reliability
- Compilation & compute tool
- Validated, benchmarked, co

How can we get the best of both worlds?

Session Topics

- **Application space** – instrumentation-centric applications (automation, test, control, measurement/monitoring)
- **Technical foundation** – hardware, software tools, AI code gen options
- **Implementation tutorial** – DEMO
- **Validation and next steps** – simulation, benchmarking, making it your own





Cyth Systems Overview

Company overview

- Headquartered in San Diego, CA with U.S.-wide coverage
- 25+ years delivering automation solutions
- Full spectrum NI Partner

Authorized Distributor

Training Partner

Integration Partner

CERTIFIED ISO 9001:2015 COMPANY

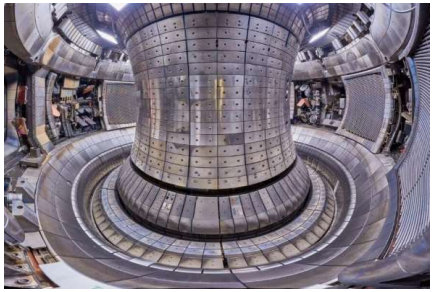
Application Expertise

- Test & measurement automation
- Precision automation & monitoring
- Embedded design & engineering
- LabVIEW & TestStand consulting services

Automation, Controls, Test

BioTech & Lab Equipment

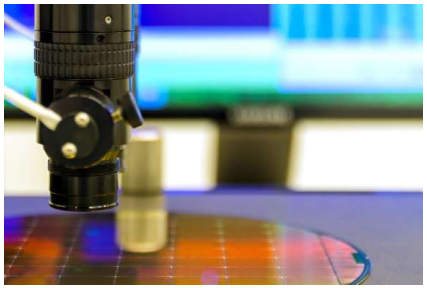
Advanced Medical Devices



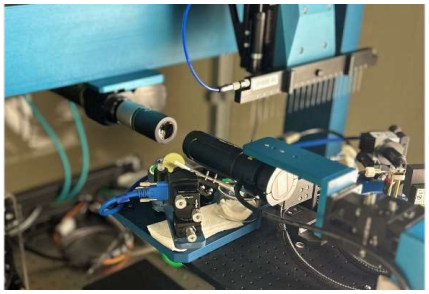
Power Gen System

Modern Grid & DER

O&G Equipment



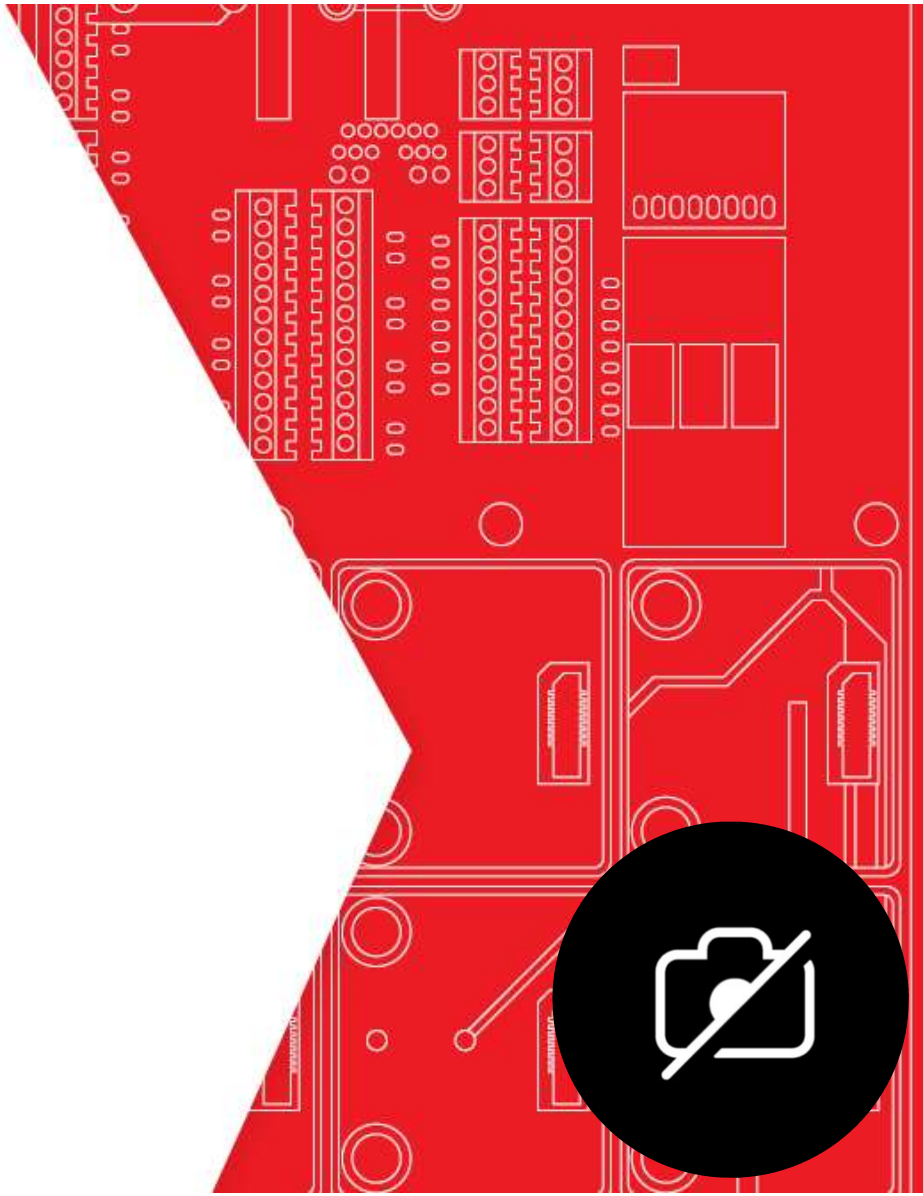
Materials Research & Fab



Precision Manufacturing

Autonomous Ma

Hardware Platforms
Software Tools
AI Code Gen Options



Platform I/O

Platform Hardware – Measure, Generate, Interface

Measure

- +/-0.25VDC up to 480VAC
- Digital (TTL, 24VDC, sinking/sourcing)
- Counters
- Current
- Temperature (TCs, RTDs)
- Load, pressure, torque sensors
- Strain gages (bridge varieties)
- Accelerometers (IEPE)
- Microphones (IEPE)
- Galvanically isolated meas.
- String pots, line pots
- Proximity probes
- pH probes
- Custom

Generate

- +/-10V output
- Current (0-20mA)
- Digital (TTL, 24VDC)
- Relay module
- Generate PWM
- Simulate sensors/signals

Interface

- RS232 / 485
- I2C/SPI
- Ethernet
- EtherCAT
- Modbus
- Custom

LabVIEW™

RIO Platform

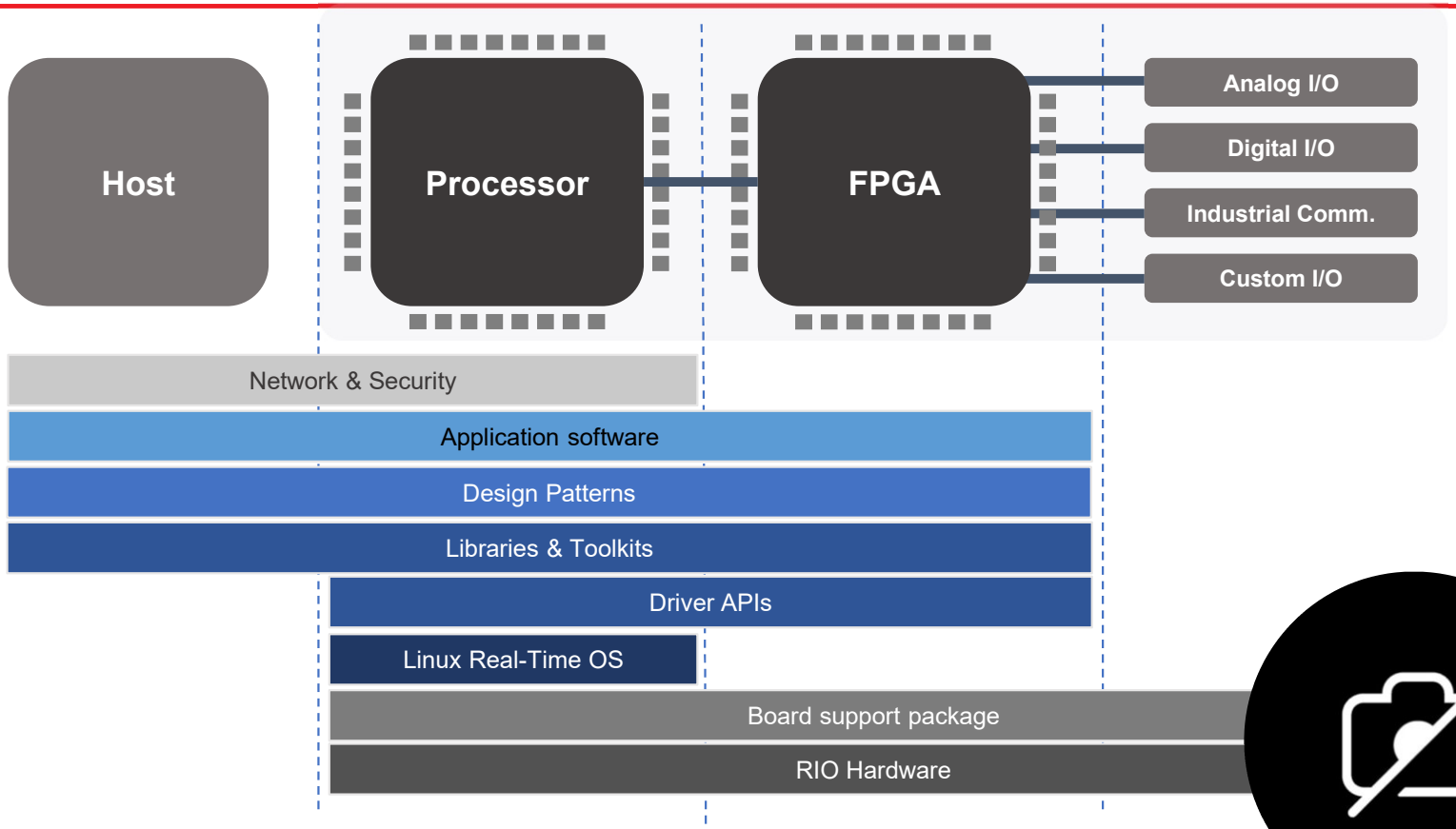
Real-Time CPU + FPGA
Versatile, heterogeneous processing

C Series I/O Modules
Measurements, Stimulus, Interfaces

Headless Auto
Ruggedized and board

Integrated Development Stack
UI to firmware with open integrations

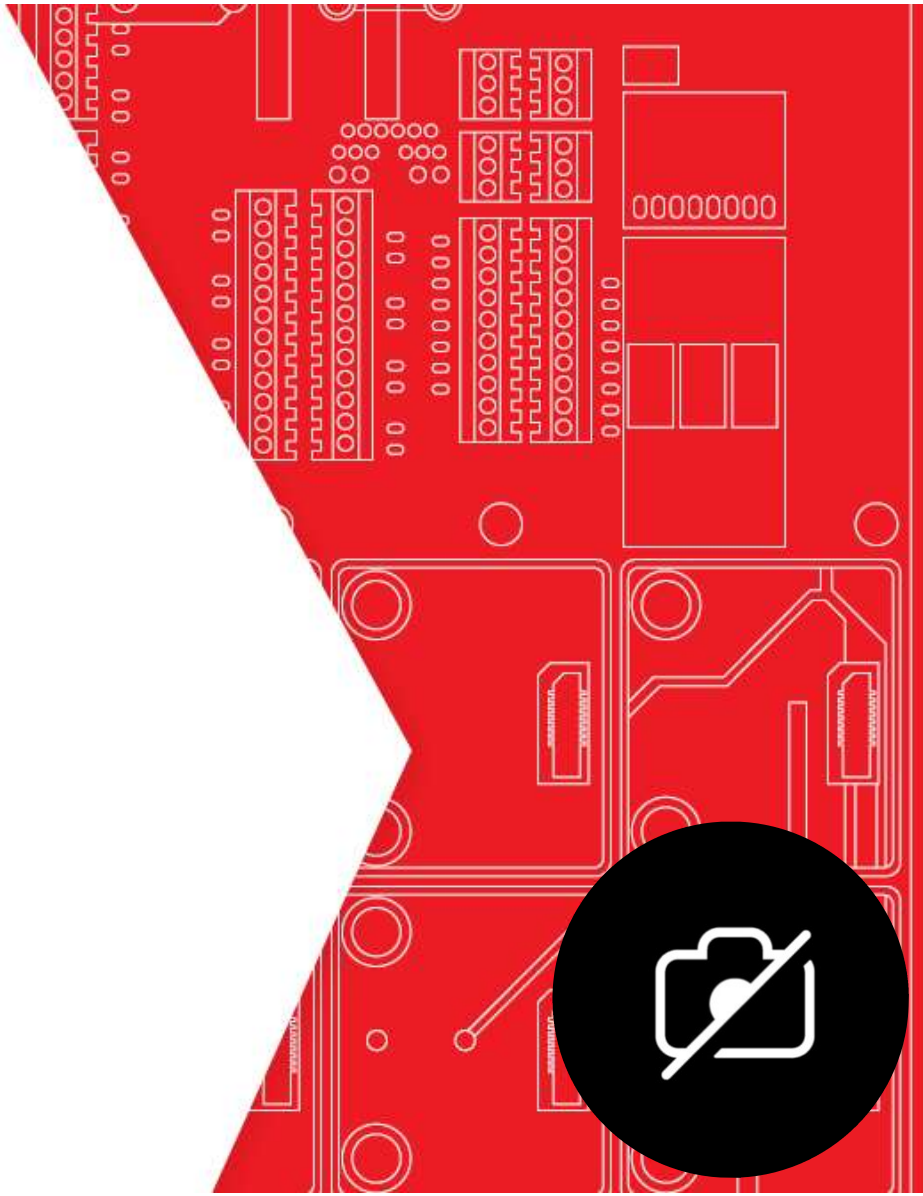
Software Tools Across the Stack



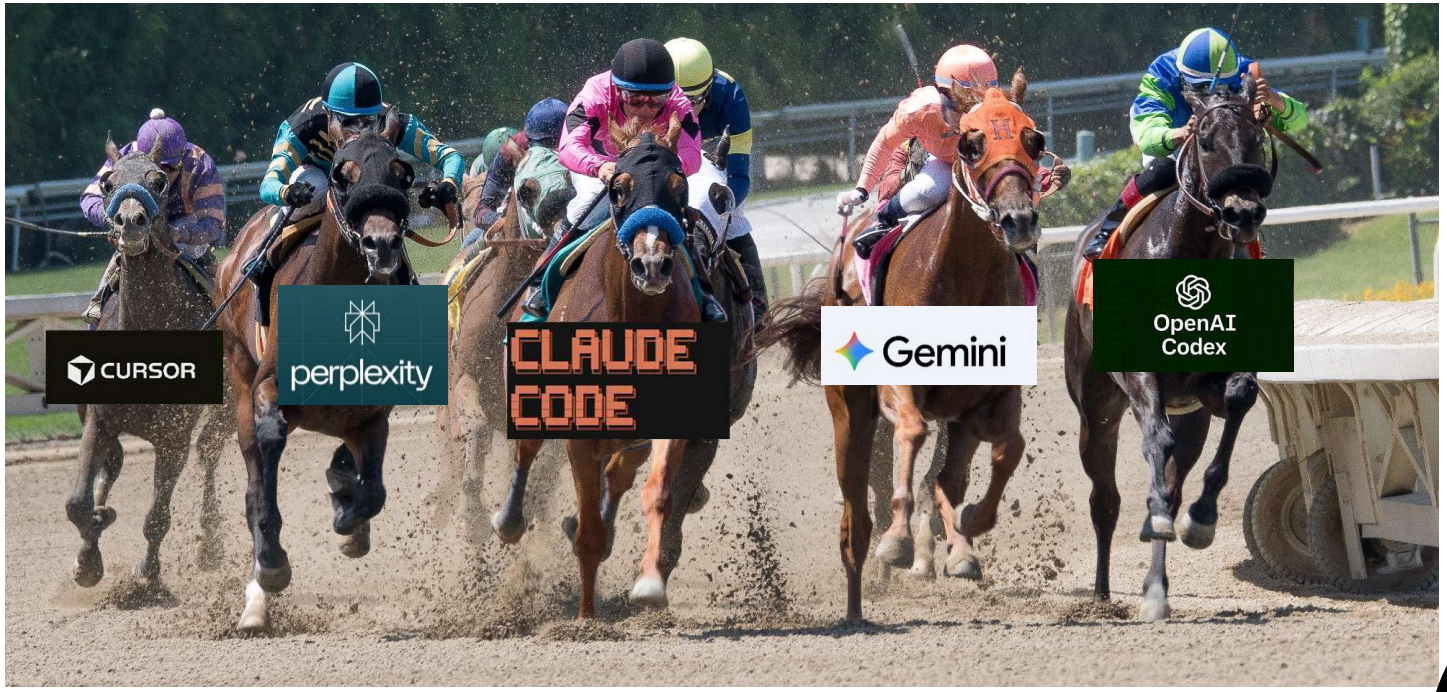
LabVIEW™

VHDL

DEMO

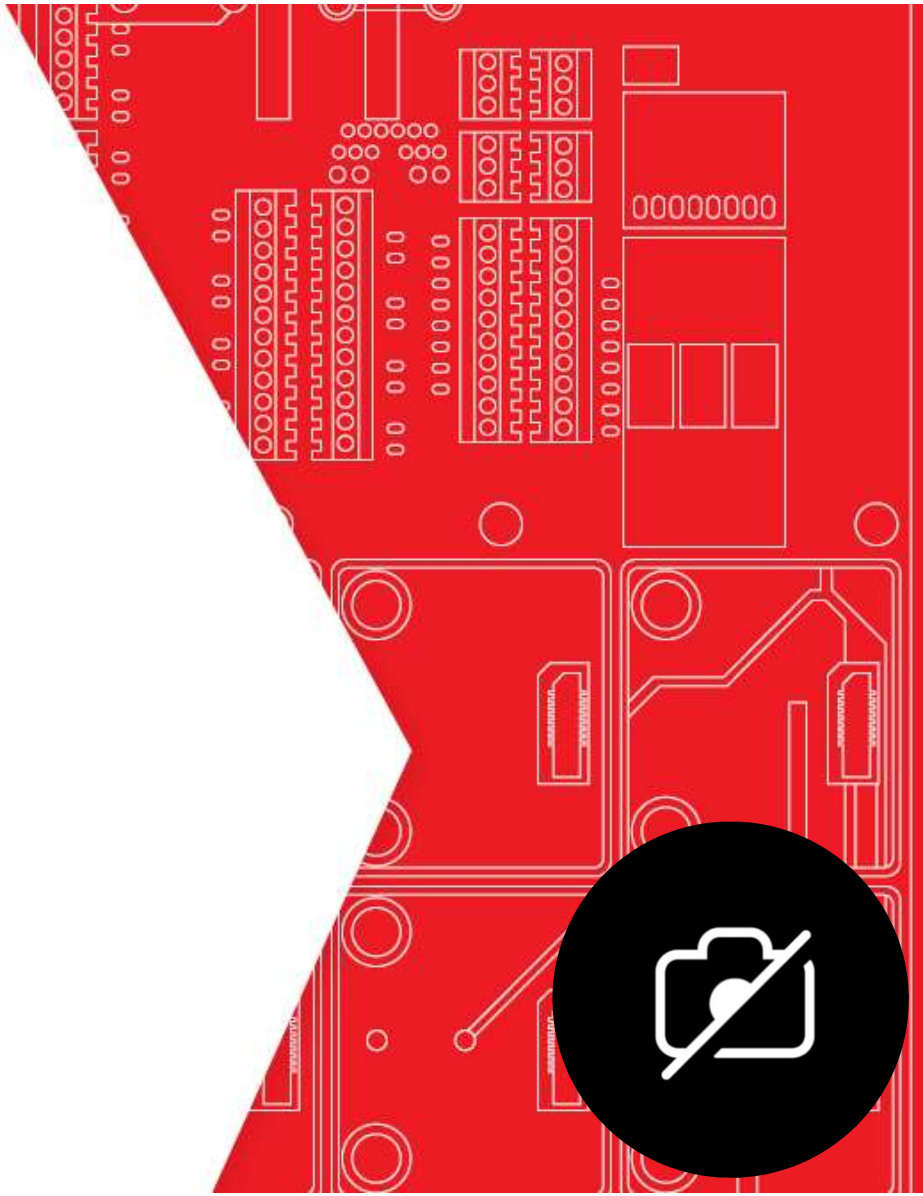


AI Code Generation Tools

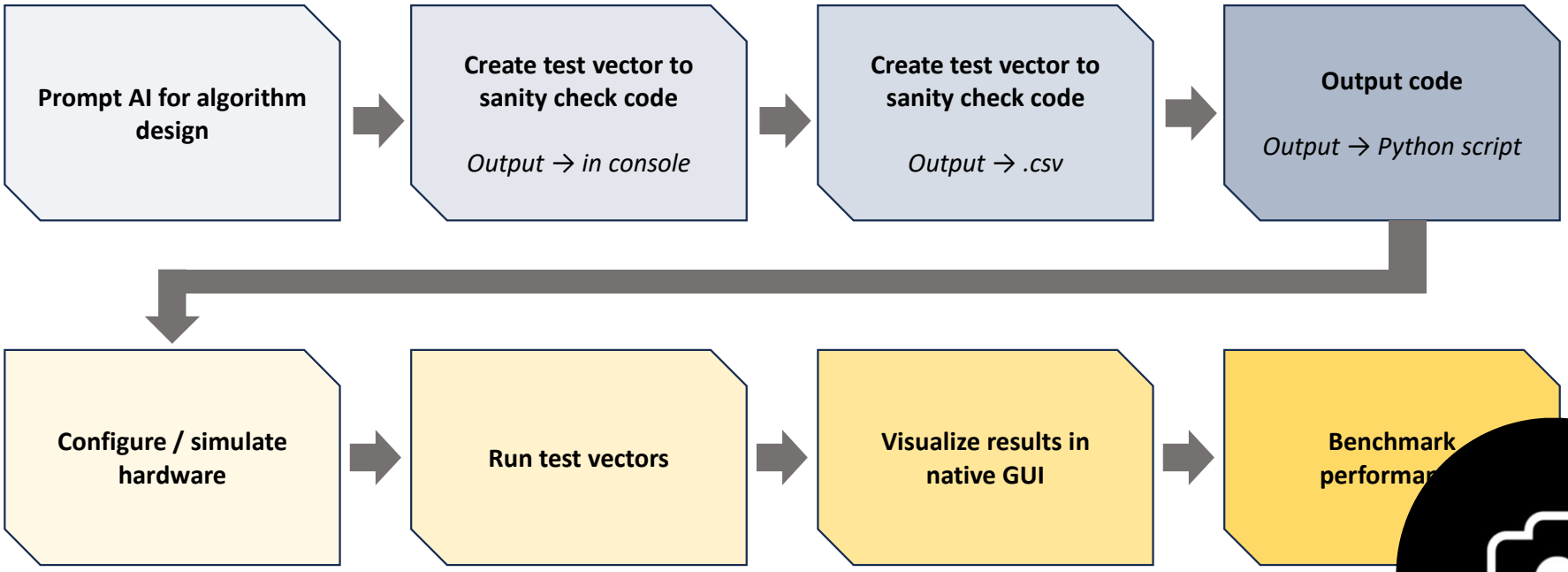


1. Learning how to prompt and build context
2. Source code control & integration
3. Test and test often

DEMO



Breaking down the workflow



LabVIEW™

Right Tool, Right Job

AI accelerator

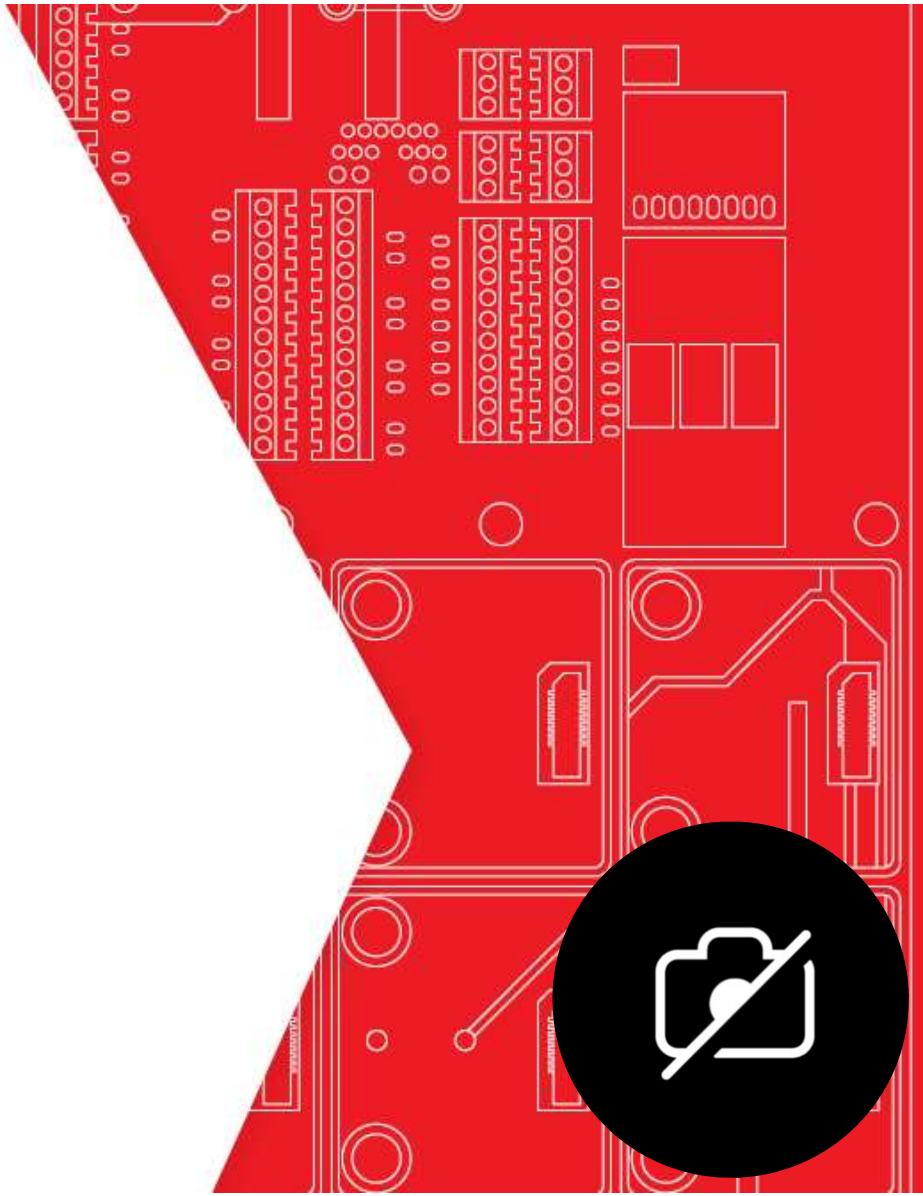
cyth systems

	VHDL	Python	C / C++ / C#	Simulink	LabVIEW
FPGA HDL	✓	-	-	-	✓
Controller I/O integration	✓	-	-	-	✓
Build & deploy to target	✓		✓		✓
Algorithm engineering	-	✓	✓	✓	✓
Test bench I/O integration	-	✓	✓	-	✓
Data processing	-	✓	✓	✓	✓
File I/O and databasing	-	✓	✓	✓	✓

Where does LabVIEW shine?

- Interactive development with rich UIs and controls
- Hardware integration and resource management
- Compilation and deployment to embedded targets
- Debugging AI code alongside real hardware
- Benchmarking hardware-based systems

DEMO

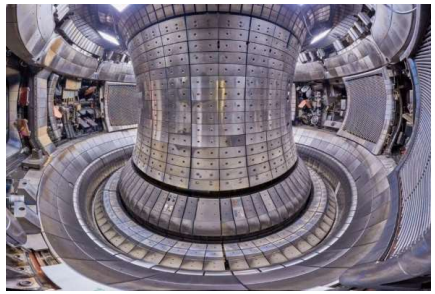


Mission-Critical Applications

Quality, Reliability, Calibration, Data Traceability, Compliance

BioTech & Lab Equipment

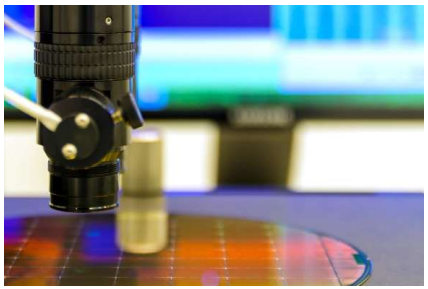
Advanced Medical Devices



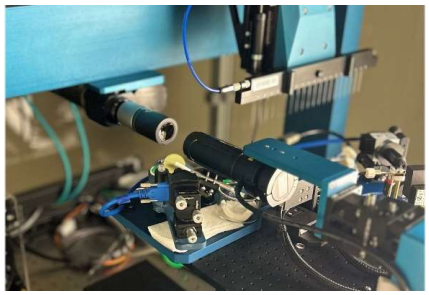
Power Gen System

Modern Grid & DER

O&G Equipment



Materials Research & Fab



Precision Manufacturing

Autonomous Mach

cyth systems

Questions?

Contact zach.olson@cyth.com

for Git repo or to discuss best practices

CASE STUDY | BIOTECH

ML-Enhanced Flow Cytometry

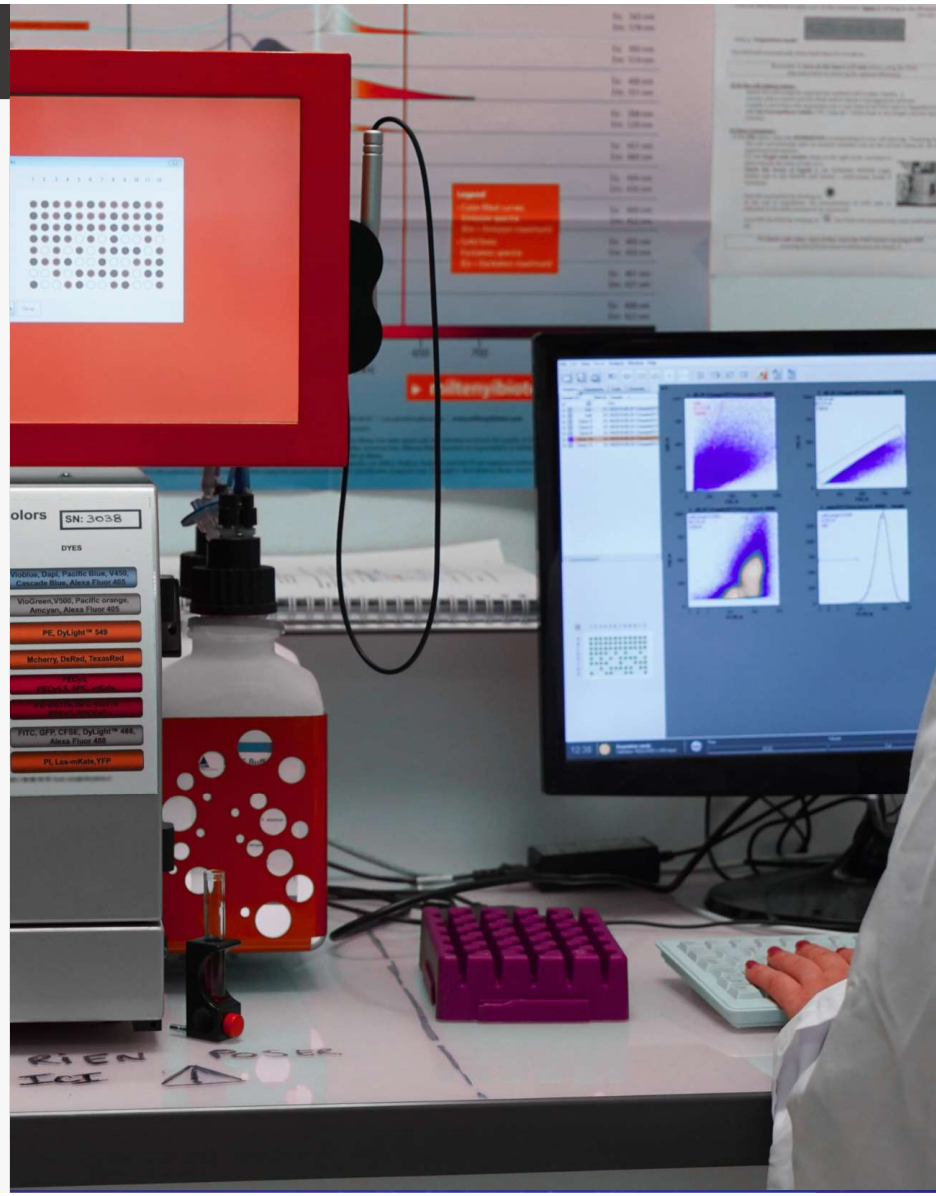
Real-time cell classification using CompactRIO and LabVIEW, enhanced by TensorFlow machine learning algorithm design and I/O-linked FPGA integration

Project Summary

- Flow cytometry requires high-speed analog pulses as cells pass a laser
- Traditional threshold logic susceptible to missing information distinguishing cell types, doublets, and debris
- Validated ML pipeline from labeled training data to deployed FPGA bitfile without manual re-gating upon changing conditions

Features & Technology

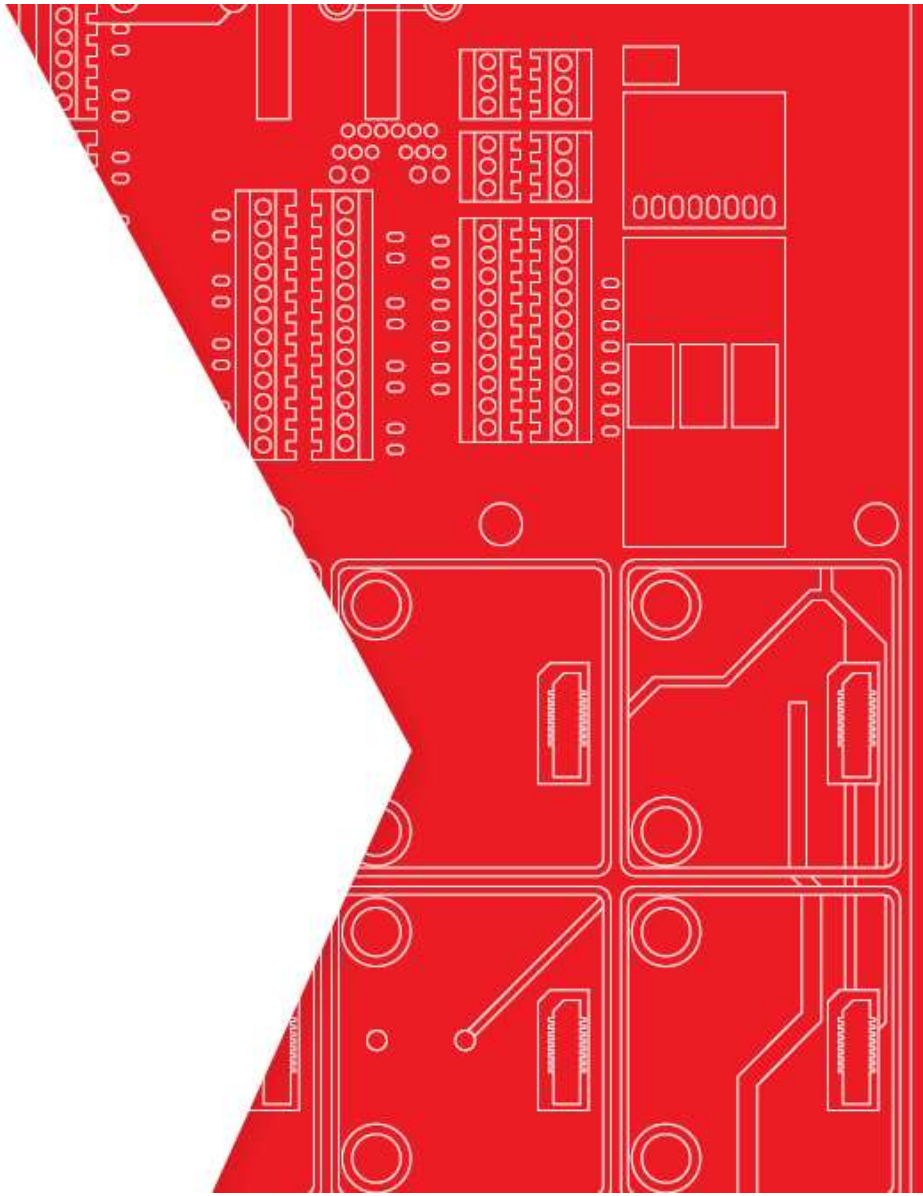
- **TensorFlow/Keras** :offline model training and validation on labeled waveform windows (64 samples/event)
- **hls4ml / DeepLTK**: converts quantized model to synthesizable HDL targeting Kintex-7 FPGA (cRIO-9040)
- **cRIO & LabVIEW FPGA**: ML block integrated as IP node and integrated with I/O for event windowing, classification, and triggering



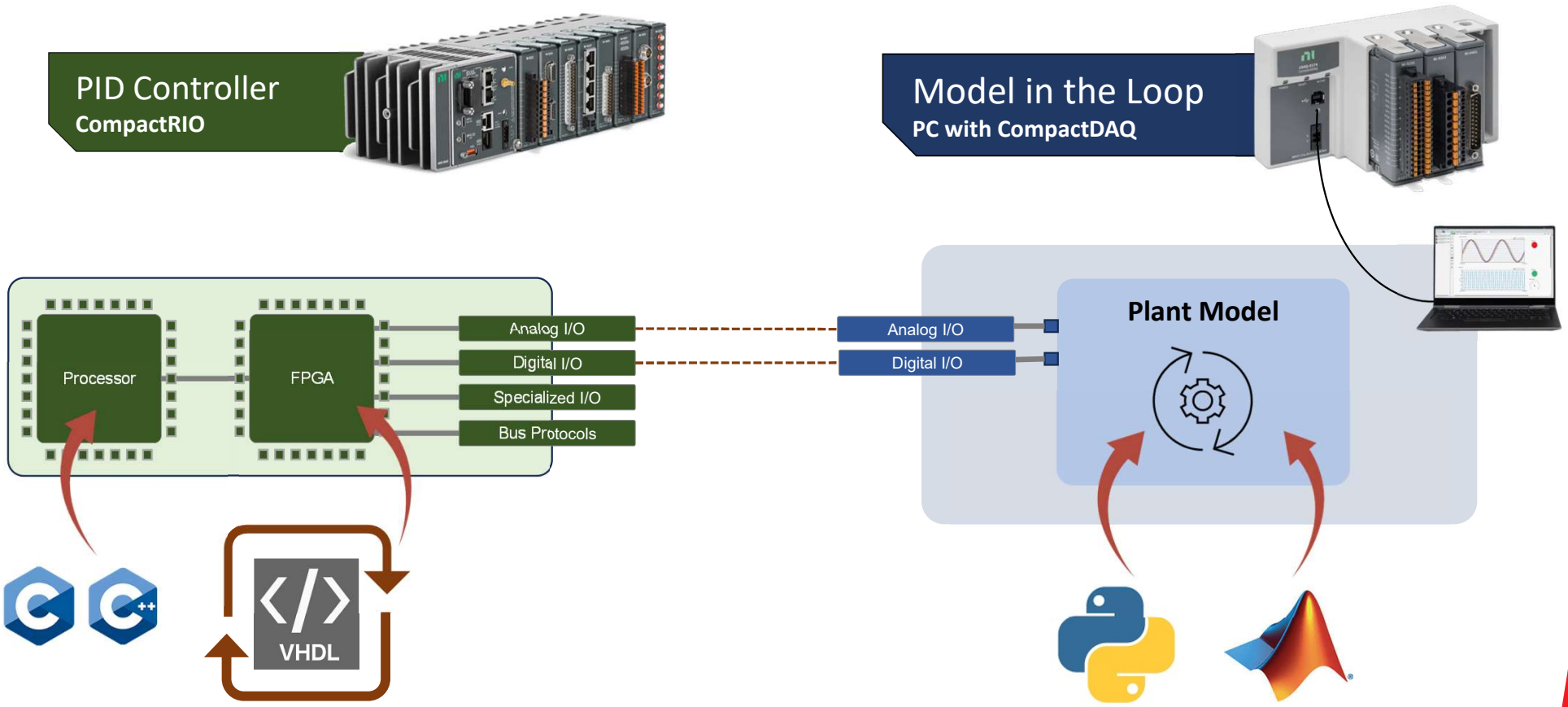
Other applications for AI generated code

- Data reduction at the edge
- Anomaly detection
- Pattern recognition
- PID tuning
- Digital twins
- Hardware-in-the-Loop

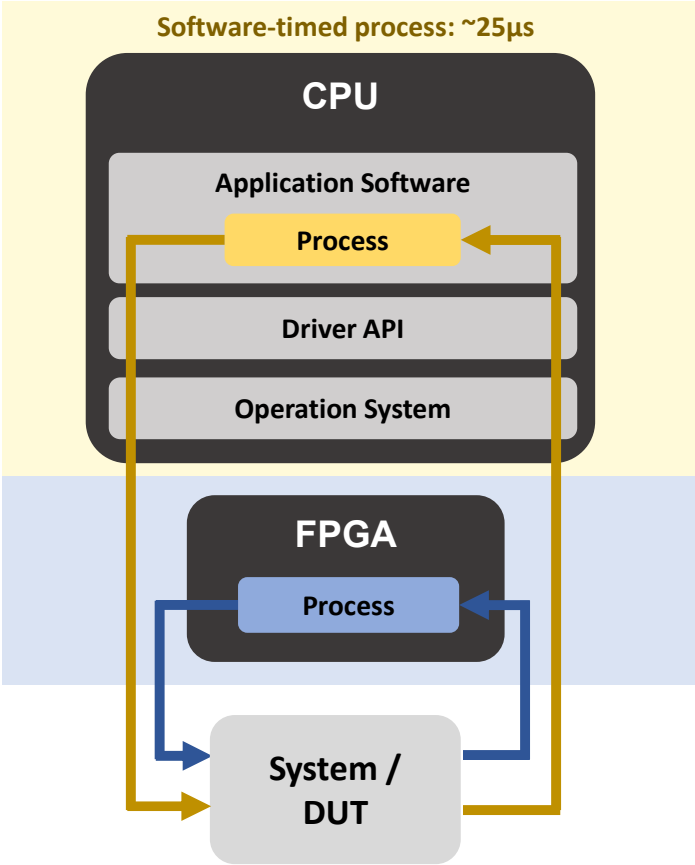
Questions?



System Diagram



Testing Controller/Tuning Model



Windows OS for File IO

LabVIEW for Model Container, DAQmx calls

Well Model

DAQmx
Comm to controller

IO to control - cDAQ
Customize responsiveness

CASE STUDY | BIOTECH

ML-Enhanced Flow Cytometry

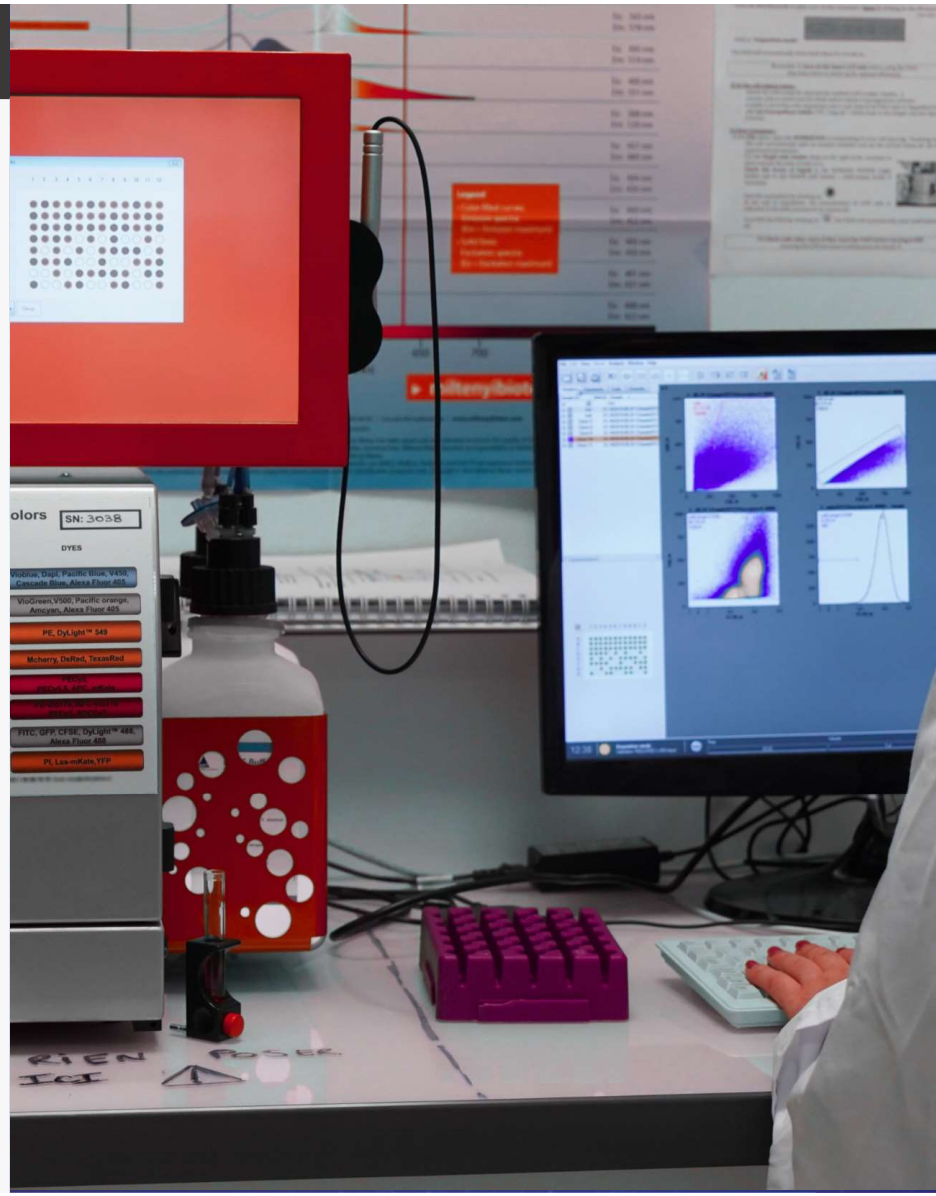
Real-time cell classification using CompactRIO and LabVIEW, enhanced by TensorFlow machine learning algorithm design and I/O-linked FPGA integration

Project Summary

- Flow cytometry requires high-speed analog pulses as cells pass a laser
- Traditional threshold logic susceptible to missing information distinguishing cell types, doublets, and debris
- Validated ML pipeline from labeled training data to deployed FPGA bitfile without manual re-gating upon changing conditions

Features & Technology

- **TensorFlow/Keras** :offline model training and validation on labeled waveform windows (64 samples/event)
- **hls4ml / DeepLTK**: converts quantized model to synthesizable HDL targeting Kintex-7 FPGA (cRIO-9040)
- **cRIO & LabVIEW FPGA**: ML block integrated as IP node and integrated with I/O for event windowing, classification, and triggering



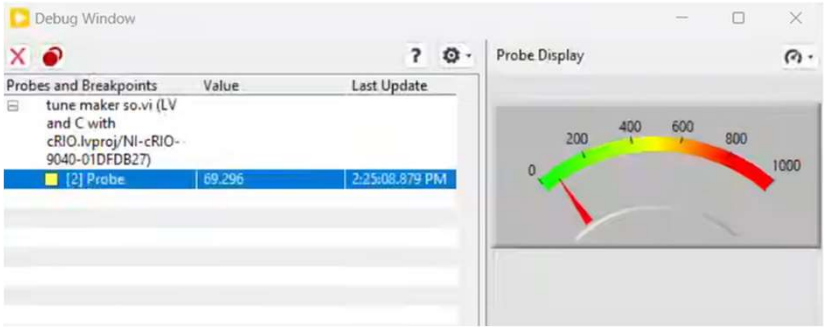
Model Generation

Software Integration testing

- Test in code gen tool (sanity tool) → create basic test vector against python code
- Generate .csv test vectors
- Export Python code and import into LabVIEW Python node
- Run test vectors in LabVIEW against simulated hardware → visualize in GUI and benchmark performance
  - Account for CPU timing, time steps, etc.

```
C:\Users\Public\Documents\GitHub\well-controller>py test_harness.py --Q_set 3.0 --mu_step 10.0 --steps 600 --log 0.0
[harness] Loaded gains from controller_gains.json: {'Kp': 1.2800302511316346, 'Ki': 0.24385365126526043, 'Kd':
0.001203351664665453, 'J': 0.005984655431123339}
[harness] Starting 600 steps | Q_set=3.0 bpm | mu=1.2->10.0 cP at step 300
[harness] Step 300: viscosity changed to 10.0 cP
[harness] Done in 0.01s (116401 steps/sec)
[harness] Results saved to run_results.csv
[harness] Running BO tuner on this episode...
[harness] BO tuner result: {'Kp': 0.846443206953178, 'Ki': 0.0031570940514950574, 'Kd': 0.028322766221534346, 'J':
0.017531114273069696}
```

LabVIEW overview



Python vs LabVIEW

- LabVIEW - IO, visualization, glue logic, hardware integration, rapid code prototyping
- Python – Machine learning, data analysis, database connectivity

Python Node Overview

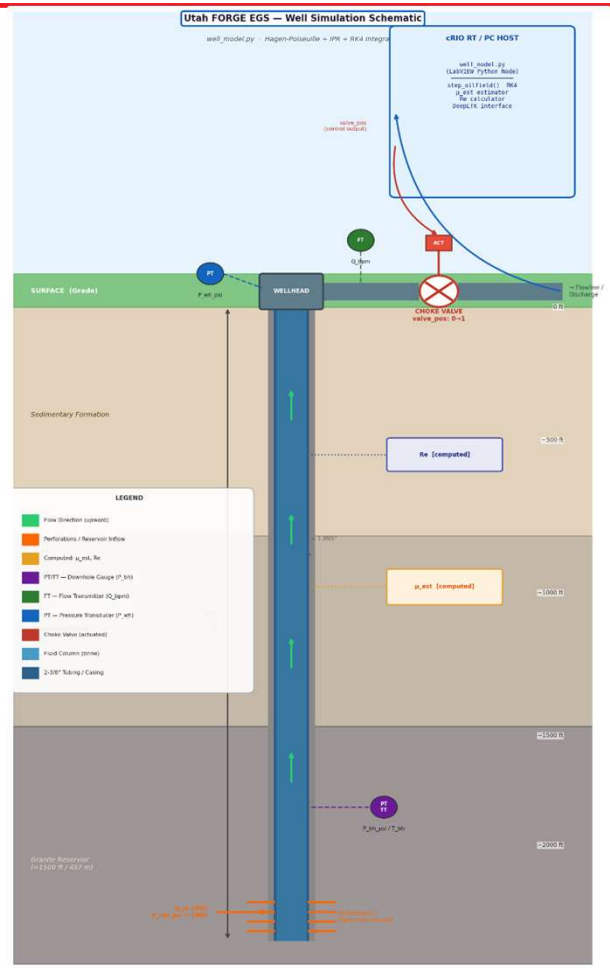
Addding advanced signal gen to LabVIEW

- Code Generation/workflow

Demo – Send sine wave to PID controller

Well Model Generation - Python

Parameter	Value	Units	Physical Meaning	Sensitivity
`mu_cp`	1.2	cP	Dynamic viscosity of brine. Water = 1.0 cP. Controls friction pressure drop via Hagen-Poiseuille $\Delta P = 8\mu LQ/\pi R^4$ .The primary adaptive signal — `mu_est` is inferred online from measured dP and Q.	**HIGH** — 10× increase → proportional Q drop
`rho_sg`	1.03	—	Fluid specific gravity vs water. Sets hydrostatic head: $P_{bh} = P_{res} - \rho g L$ .FORGE brine is slightly denser than fresh water due to dissolved salts.	LOW
`Q_set_bpm`	3.0	bbl/min	Flow rate setpoint commanded by controller. This is what the PID or DeepLTK is trying to achieve by adjusting `valve_pos`. gdr.openei+1	Control input
`R_in`	1.995	inches	**Inner radius** of tubing. Enters friction as R^4 —a 10% error in R causes 46% error in ΔP_{fric}-2 . "3/8OD tubing (1.995" ID) is standard for FORGE-depth EGS completions.	**CRITICAL** — R^4dependence
`L_ft`	1500.0	ft	True vertical depth to perforation midpoint. Sets both hydrostatic head and total tubing friction path. FORGE Phase 1 target zone	Moderate



Benchmarking & Advanced Topics

- Determinism, latency
- LLM prompting and assisted traceability
- Source code control

Conclusion

- Next steps

Project Scoping

Phase 1 (RIO running against simulated model)				
Task	**Subtask**	**Hours**	**Owner**	**Notes**
Feasbility of VHDL code gen		**8**	Zach	Can we trust AI judgement on models that fit in LV FPGA constraints, Can we constrain code gen?
Integration of AI gen VHDL in CLIP node		**8**	Zach	Completed, need to document
Validation test bench for VHDL in clip		**18**	Cyth Engineer	DMA FIFOs, channel interpretation
	DMA FIFOs	4	Cyth Engineer	
	Data Logging	4	Cyth Engineer	
	Automation - reset model and run time steps	10	Cyth Engineer	
Visualize a specific use case		**16**	Cyth Engineer	
	UI for valve and orifice temp, pressure, flow - graph and numeric	8	Cyth Engineer	
	Playback features for recorded data	8	Cyth Engineer	
Desktop model		**16**		
	Model selection	8	Zach	
	Model implementation	8	Zach	
Iterate on model and test bench		**8**		
Power Point Presentation		**8**	Quentin	
PM		**8**	Quentin	

Phase 2: Physical demo (e.g., real valve) / CFX
Phase 3: Highly portable

NI Connect Session

Session Title: Adding AI and ML to Your CompactRIO Software Development Workflow

Session goals: Learn tips for using AI tools like Claude and Perplexity to develop cRIO systems. See a demo of a machine learning (ML) valve controller with code deployed to the FPGA through a LabVIEW I/O Node Interface for CLIP.

- Generate VHDL → using CLIP nodes → compile → deploy → validate
  - Use off-the-shelf valve model (Python, Simulink) driving cDAQ I/O?
  - Embedded code running on cRIO

Out of scope:

- Generate model in VHDL → compile → deploy → validate

cyth systems